

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

**«Санкт-Петербургский политехнический университет Петра
Великого»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

02.03.01 Математика и компьютерные науки

Отчёт
Современные технологии разработки программного
обеспечения
Отладка и анализ ошибок в C++

Выполнил:

студент группы 5130201/50301 _____ Ташлыкова Е.В.

Принял:

Ассистент _____ Кирпиченко С.Р.

«____» _____ 2026г.

Санкт-Петербург, 2026

Аннотация

Данная работа посвящена исследованию типовых ошибок в языке программирования C++ и методов их обнаружения с использованием современных инструментов отладки и анализа.

Цель работы — систематизация знаний о причинах возникновения типовых ошибок в C++ и анализ различных инструментов их обнаружения.

Методы: воспроизведение минимальных примеров ошибочного кода, динамический анализ с использованием Valgrind, AddressSanitizer (ASan), UndefinedBehaviorSanitizer (UBSan), статический анализ с помощью clang-tidy.

Результаты: приведены и проанализированы причины возникновения segmentation fault (в том числе частные случаи UB), утечки памяти, ошибки линковки, undefined behavior; приведены примеры обнаружения ошибок.

Область применения: результаты могут быть использованы в учебном процессе и практической разработке на C++ для повышения надёжности ПО.

Содержание

Введение	4
1 Segmentation fault	5
2 Утечка памяти	8
3 Ошибка линковки	10
4 Undefined behaviour	12
Заключение	13
Список используемых источников	14

Введение

Язык C++ предоставляет разработчику высокий уровень контроля над памятью и аппаратными ресурсами, однако это накладывает дополнительную ответственность на программиста за корректное управление ресурсами. Ошибки в работе с памятью, некорректные операции и проблемы линковки могут приводить к аварийным завершениям программ, утечкам ресурсов и неопределённому поведению.

Для обнаружения подобных ошибок существуют специализированные инструменты:

- Статический анализ (clang-tidy, PVS-Studio): проверяет исходный код без выполнения.
- Динамический анализ (Valgrind, AddressSanitizer, UndefinedBehaviorSanitizer): отслеживает ошибки во время работы программы.

1 Segmentation fault

Ошибка, которая возникает во время обращения программы к участку памяти, доступ к которой ей запрещен или которая не существует.

Частые причины возникновения ошибки:

- Получение доступа к памяти, которой программа не владеет.
- Разыменовывание nullptr указателя.
- Изменение памяти доступной только для чтения.

В стандарте языка C++ отсутствует понятие «segmentation fault». Большинство действий, приводящих к этой проблеме, согласно стандарту C++, являются неопределённым поведением (Undefined Behaviour, UB). Segmentation fault – это одно из возможных проявлений UB на платформах с менеджером памяти и защитой страниц.

1.1 Изменение строкового литерала

```
int main()
{
    char* str;
    str = "hello";
    *(str + 1) = 'n'; // Error
    return 0;
}
```

Проблема возникает потому что мы пытаемся записать данные в память, доступную только для чтения. Попытка записи – UB, на практике – сегфолт.

1.2 Обращение к освобожденной памяти

```

int main()
{
    int n = 10;
    char *str = new char[n];
    str = "hello";
    delete str;
    *str = "world"; // Error
    return 0;
}

```

Проблема возникает потому что, указатель `str` разыменовывается после освобождения блока памяти, что недопустимо с точки зрения компилятора. Это UB, часто — сегфолт или повреждение данных.

1.3 Разыменовывание нулевого указателя

```

#include <iostream>
int main()
{
    int* ptr;
    std::cout << *ptr; // Error
    return 0;
}

```

Происходит обращение к неизвестному участку памяти, что приводит к ошибке сегментации или к другому неопределенному поведению.

1.4 Выход за границы массива

```

int main()
{
    int arr[2];

```

```
arr[3] = 10; // Error  
return 0;  
}
```

Обращение к элементу массива по индексу, превышающему его размер, приводит к чтению или записи памяти, не принадлежащей массиву. Из-за этого могут возникнуть ошибки сегментации или другое неопределенное поведение.

Обнаружить ошибки можно с помощью AddressSanitizer:

```
g++ -fsanitize=address -g example.cpp -o example  
./example
```

ASan поймает выход за границы, use-after-free и разыменование нуля, указав точную строку.

2 Утечка памяти

Это ситуация, когда память, выделенная для выполнения определенной задачи, остается выделенной даже после того, как необходимость в ней отпадает. Это приводит к нерациональному использованию памяти, поскольку она недоступна для других задач до завершения работы программы. Так как в C++ нет автоматических сборщиков мусора любое выделение и освобождение динамической памяти становится ответственностью программиста.

```
void f()
{
    int* ptr = new int[10]; // Выделяем память
    // Не освобождаем ее
    return;
}
int main()
{
    f();
}
```

Выделенная память перестает быть доступной, но при этом остается занятой. Со временем, если программа продолжает расходовать память, то система начнет работать медленнее, появятся другие ошибки, а память, так как это ограниченный ресурс, закончится.

Для обнаружения утечек памяти существуют такие инструменты, как Valgrind и AddressSanitizer.

Valgrind — это фреймворк для отладки и профилирования памяти. Он может обнаруживать утечки памяти, некорректные обращения к памяти и другие ошибки. Valgrind предоставляет подробные отчеты об утечках памяти, включая информацию об их происхождении и возможных причинах.


```
# установка
brew install valgrind

# компиляция с отладочной информацией и запуск
g++ -g -O0 main.cpp -o programm
valgrind --leak-check=full ./programm
```

AddressSanitizer, также известный как (Asan), — это быстрый детектор ошибок в работе с памятью, который встроен в компиляторы GCC и Clang.

```
g++ -fsanitize=address -g -O0 main.cpp -o programm && ./programm
# или
clang++ -fsanitize=address -g -O0 main.cpp -o program ./programm
```

Эти инструменты имеют свои отличия.

Например, **скорость:** ASan работает значительно быстрее благодаря тому, что он внедряет проверки непосредственно в код на этапе компиляции, программа замедляется, по сравнению с обычным запуском. Valgrind же эмулирует выполнение каждой инструкции процессора, что приводит к большему замедлению.

Потребление памяти: Valgrind потребляет умеренное количество дополнительной памяти, тогда как ASan требует значительно больше ресурсов, потому что создаёт "теневую память"(shadow memory) для отслеживания состояния каждого байта. **Необходимость перекомпиляции:** Valgrind работает с уже скомпилированным бинарным файлом, достаточно запустить программу под Valgrind. ASan же требует перекомпиляции всего проекта со специальными флагами компилятора '-fsanitize=address -g'.

Таким образом для CI/CD и ежедневной разработки лучше подходит ASan (быстрее). Для финального анализа сложных багов или при невозможности перекомпиляции – Valgrind.

3 Ошибка линковки

Связывание - это процесс объединения отдельных объектных файлов и библиотек в одну исполняемую программу. Ошибки связывания возникают, когда компоновщик не может разрешить ссылки между различными объектными файлами или библиотеками. Эти ошибки препятствуют созданию конечного исполняемого файла.

Частые причины возникновения ошибок линковки - это отсутствие реализации функции, некорректные пути к библиотекам, несовпадение сигнатур функций и циклические зависимости.

```
// header.hpp
int calculate(int x);

// main.cpp
#include "header.hpp"
int main()
{
    int result = calculate(10); // Error
    return 0;
}
```

Возникает ошибка - неопределенная ссылка на calculate. Компоновщик не может найти её тело.

```
void print_val(int x) {}
void print_val(double x);

int main()
{
    print_val(3.14); // Error
    return 0;
}
```

```
}
```

Мы хотим использовать функцию с другим типом параметра. Компоновщик ищет функцию с сигнатурой `print_val(double)`, но существует только `print_val(int)`.

Для обнаружения ошибок можно использовать флаги компилятора и компоновщика:

```
gcc -Wall -Wextra -Werror main.c -o program
```

4 Undefined behaviour

Неопределённое поведение — это ситуации, при которых стандарт языка C++ не определяет ожидаемое поведение программы. Компилятор может сгенерировать любой код, программа может работать некорректно, падать или делать непредсказуемые вещи.

```
// 1. Использовать объект после move
```

```
std::string x = "hello";  
std::string y = std::move(x);  
std::cout << x; // Error
```

```
// 2. Деление на ноль
```

```
int n = 0;  
return n / 0; // Error
```

```
// 3. Выход за пределы индексирования
```

```
int arr[4] = {0, 1, 2, 3};  
return arr[5]; // Error
```

Проблемы можно обнаружить с помощью UndefinedBehaviorSanitizer (UBSan):

```
g++ -fsanitize=undefined -g example.cpp -o example  
./example
```

Однако, UBSan может не поймать ошибку при использовании объекта после move, так как формально чтение перемещённого объекта не всегда является UB (зависит от типа). Для таких случаев рекомендуется статический анализ. Например, с помощью clang-tidy:

```
clang-tidy example.cpp --checks='*' --
```

Заключение

В ходе выполнения работы были рассмотрены четыре основных типа ошибок в языке C++: ошибка сегментации (Segmentation fault), утечка памяти, ошибка линковки и неопределённое поведение (Undefined behaviour). Для каждого типа были приведены минимальные примеры кода, воспроизводящие соответствующую проблему, и продемонстрированы способы их обнаружения с использованием современных инструментов анализа.

Основные выводы:

1. **Segmentation fault** возникает при попытке доступа к памяти, которая не принадлежит программе или доступна только для чтения. AddressSanitizer эффективно обнаруживает такие ошибки, указывая точное место нарушения.

2. **Утечка памяти** происходит, когда динамически выделенная память не освобождается после её использования. Valgrind позволяет точно идентифицировать утечки с указанием места выделения памяти.

3. **Ошибка линковки** проявляется на этапе сборки программы и связана с неразрешёнными ссылками между объектными файлами. Флаги компилятора ‘-Wall -Wextra -Werror’ помогают выявить многие проблемы на ранних стадиях.

4. **Неопределённое поведение** — наиболее опасный тип ошибок, так как программа может работать некорректно без видимых сообщений. UndefinedBehaviorSanitizer позволяет выявить многие случаи UB, включая деление на ноль и использование перемещённых объектов.

5. **Статический анализ** (clang-tidy) является обязательным инструментом в современной разработке, так как позволяет находить ошибки без выполнения программы, что особенно необходимо для крупных проектов.

Список используемых источников

- [1] Дьюхэрст С. Скользкие места C++. Как избежать проблем при проектировании и компиляции ваших программ / С. Дьюхэрст. — Москва : ДМК Пресс, 2006. — 264 с. — ISBN 5-94074-083-9.
- [2] GeeksforGeeks. "Segmentation Fault in C/C++"[Электронный ресурс]: <https://www.geeksforgeeks.org/segmentation-fault-c-cpp/> (дата обращения: 10.05.2026).
- [3] GeeksforGeeks. "Memory Leak in C++ and How to Avoid It"[Электронный ресурс]: <https://www.geeksforgeeks.org/memory-leak-in-c-and-how-to-avoid-it/> (дата обращения: 10.05.2026).
- [4] GeeksforGeeks. "Detect Memory Leaks in C++"[Электронный ресурс]: <https://www.geeksforgeeks.org/detect-memory-leaks-in-cpp/> (дата обращения: 11.05.2026).
- [5] LabEx. "How to Handle Linking Errors Effectively"[Электронный ресурс]: <https://labex.io/ru/tutorials/c-how-to-handle-linking-errors-effectively-419526> (дата обращения: 11.05.2026).
- [6] GeeksforGeeks. "Undefined Behavior in C/C++"[Электронный ресурс]: <https://www.geeksforgeeks.org/undefined-behavior-c-cpp/> (дата обращения: 12.05.2026).
- [7] Clang-Tidy — инструмент статического анализа для C++ [Электронный ресурс]: <https://clang.llvm.org/extra/clang-tidy/> (дата обращения: 12.05.2026).